Advanced Software Development

Assignment#3: Food Ordering System

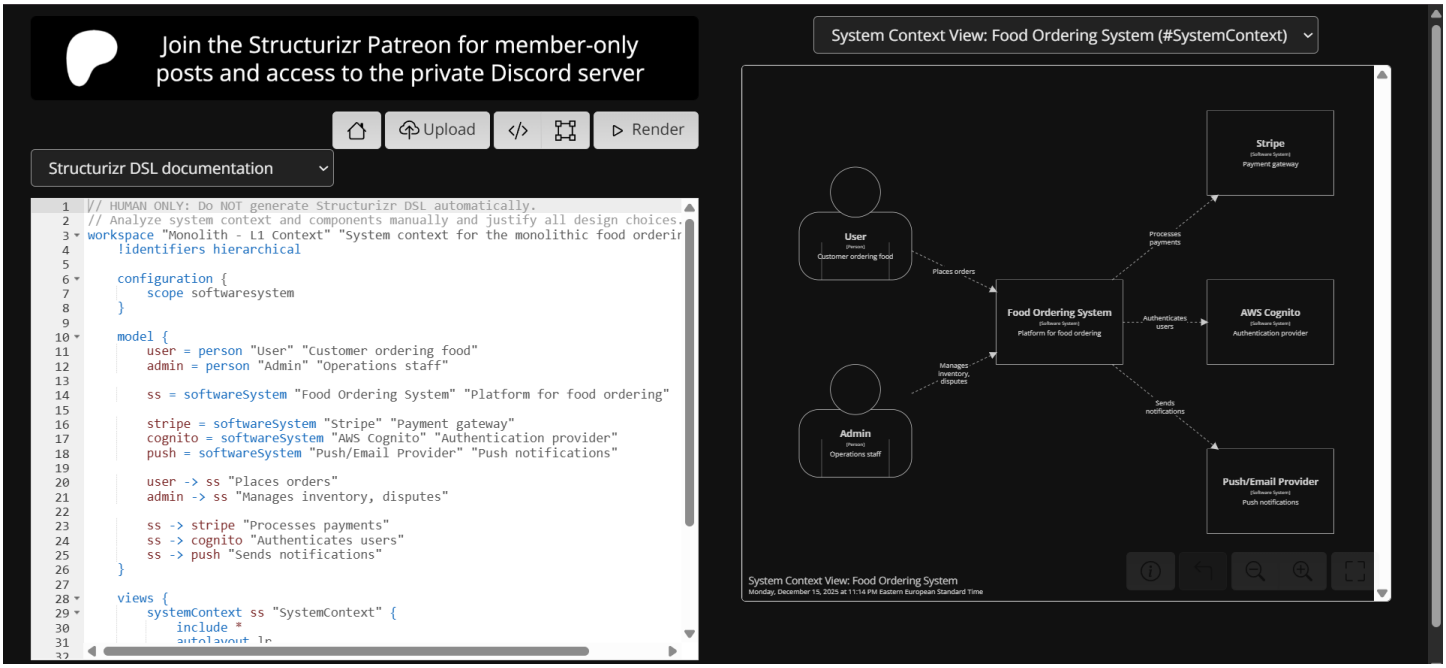
Instructor: Mustafa Assaf

Hamza Nasser

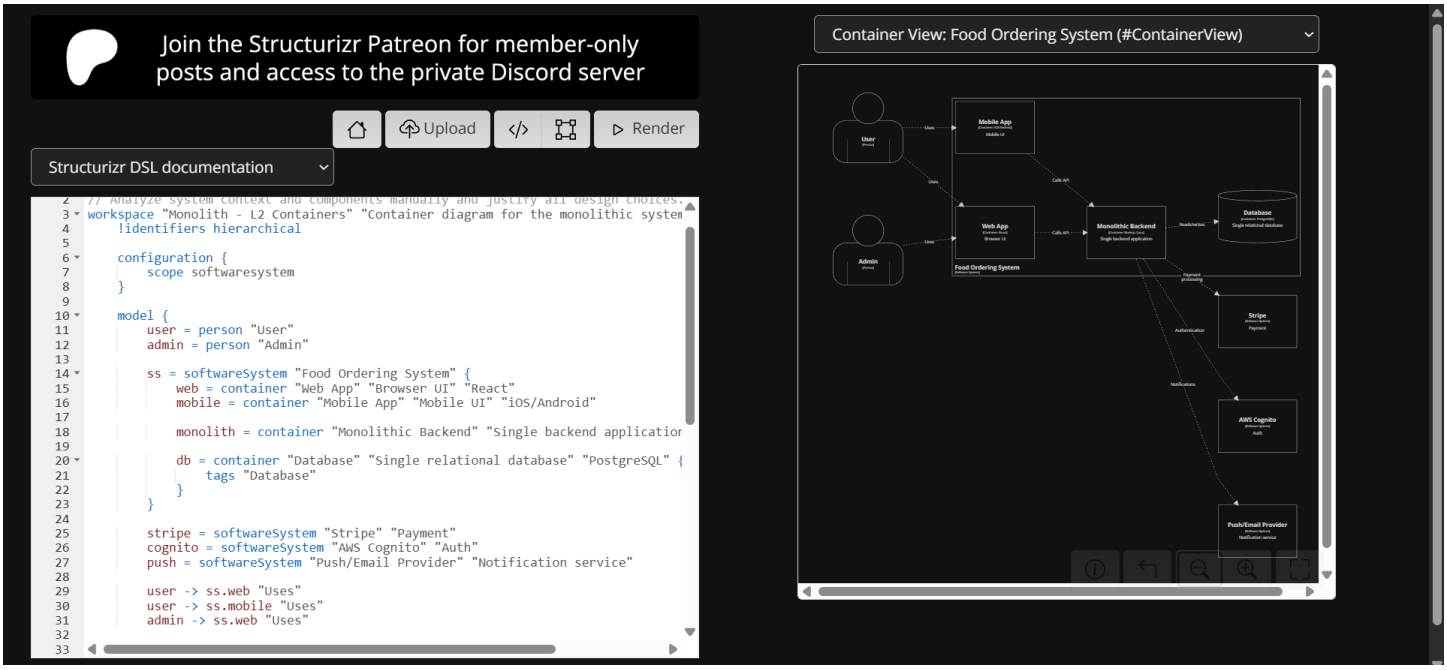
12324183

Applying **DSL source code** and running it only:

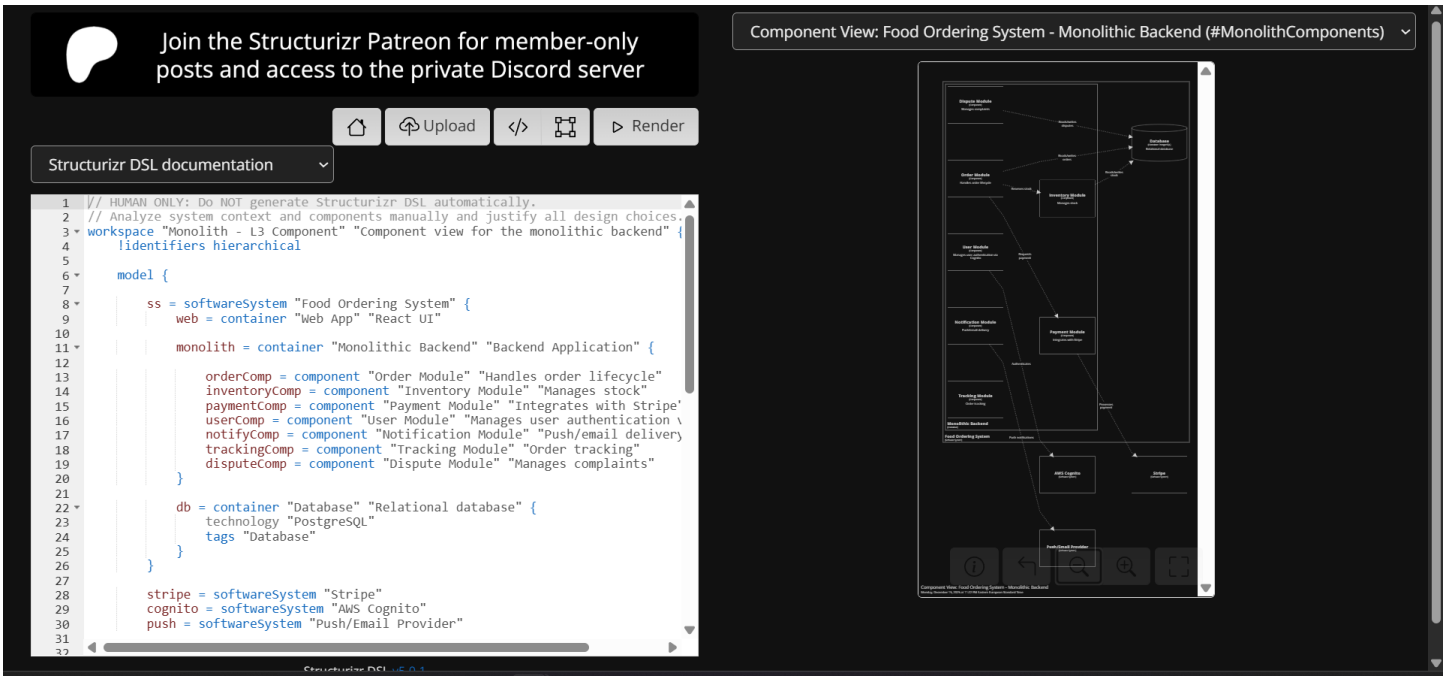
Monolith-Level - C1



Monolith-Level - C2

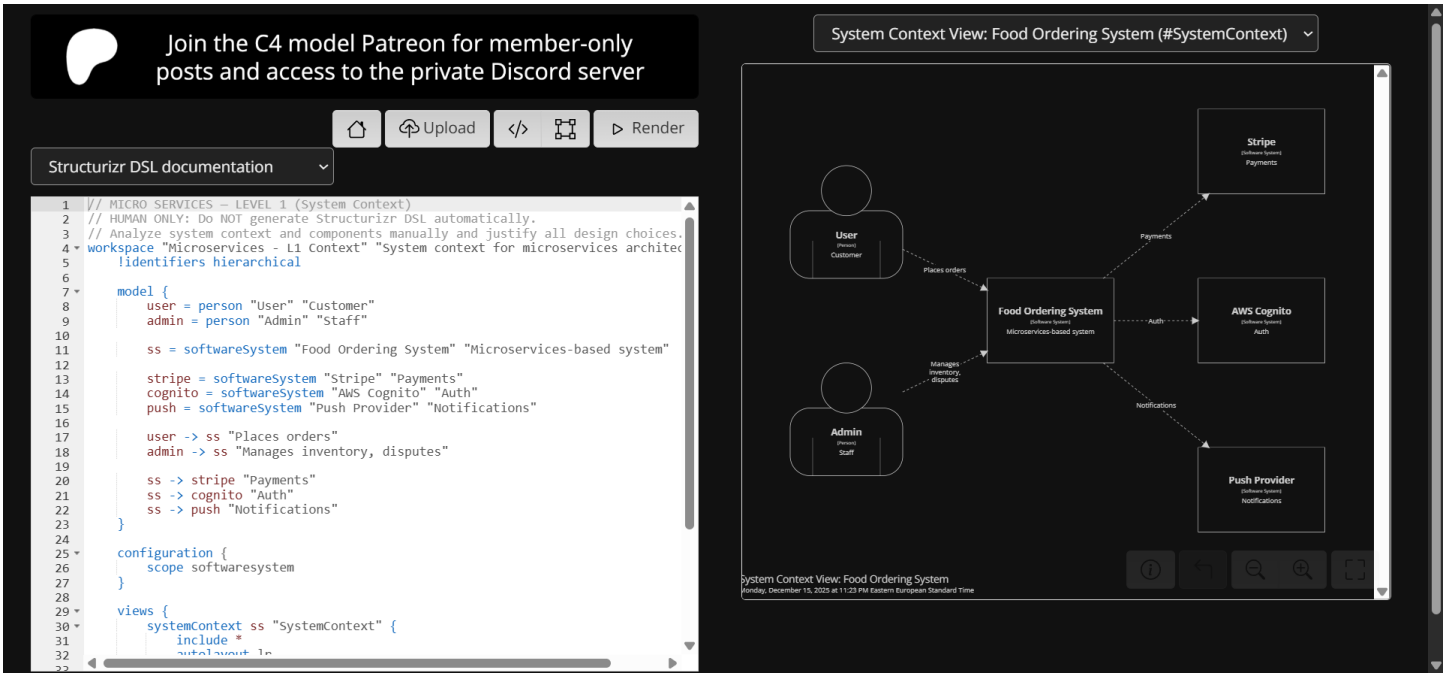


Monolith-Level – C3

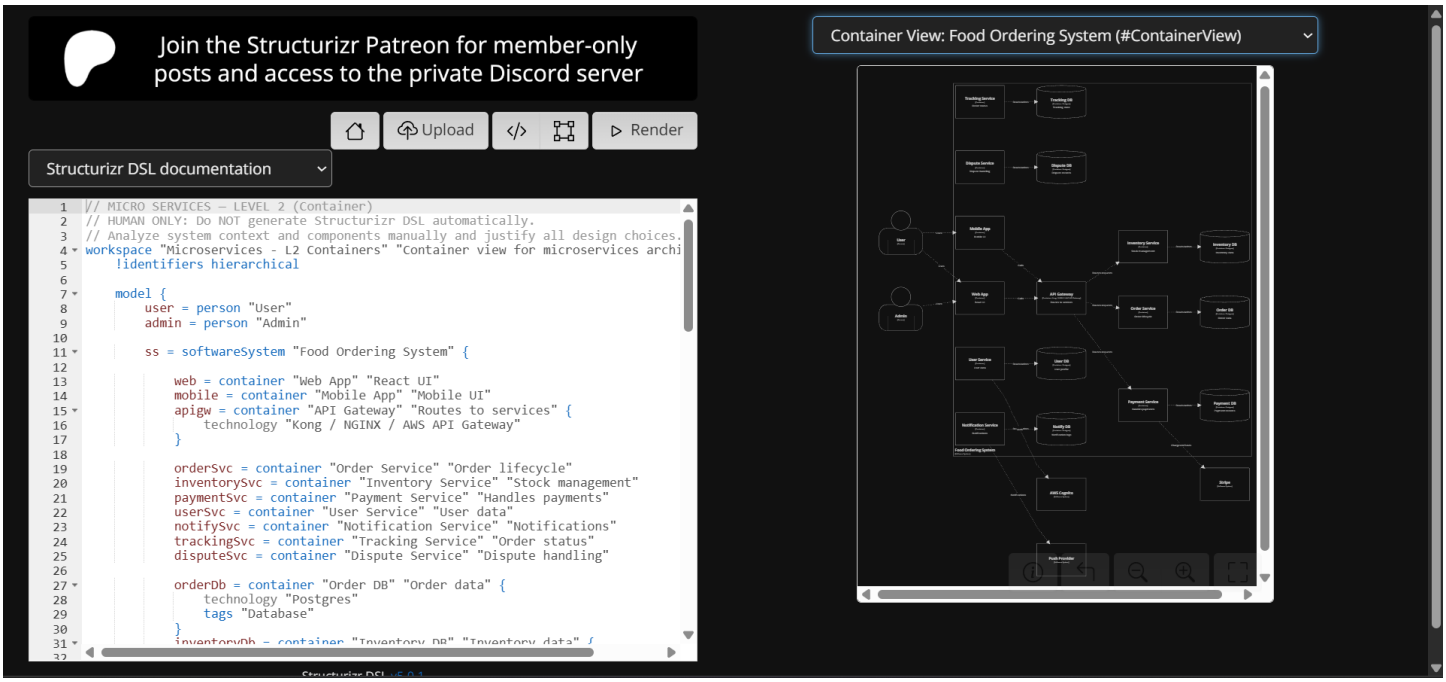


Now it's for Microservices DLS code

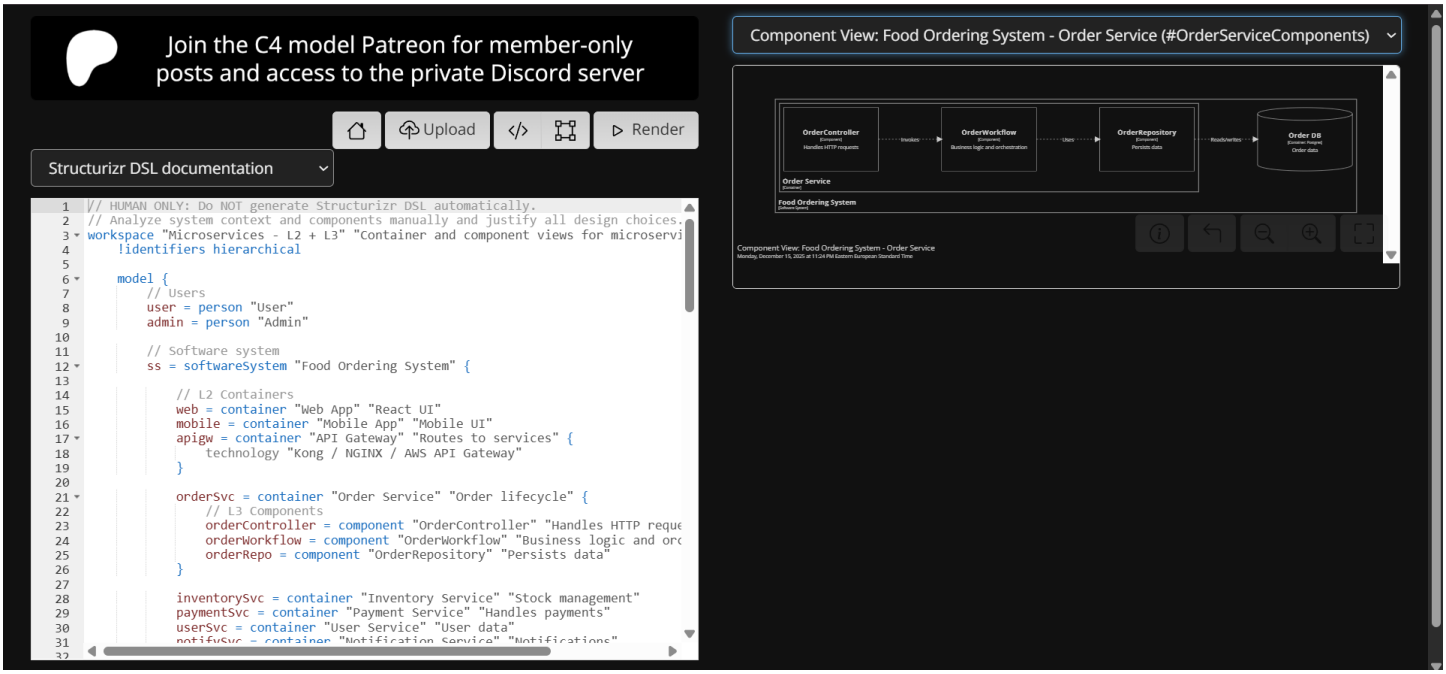
Microservices Level – C1



Microservices Level – C2



Microserveces Level – C3



Let's go to Observations and questions:

Level 1 – System Context (C4 L1)

Observation / Analysis Questions

1. Who are the main actors (users, admins, external systems) interacting with the Food Ordering System?

Main Actors:

- **User (Customer):** The primary actor who orders food, tracks deliveries, and manages disputes.
- **Admin:** Manages inventory levels, handles dispute resolution, and watches system operations.

External Systems:

- **Stripe:** Payment gateway that processes credit card transactions and refunds.
- **AWS Cognito:** Authentication provider that manages user login, registration, and session management.
- **Push/Email Provider:** Notification service (FCM/APNs/SES) that delivers order confirmations and status updates.
- **Loyalty Provider (NEW):** Tracks user reward points and provides loyalty-based discounts.

2. Which external systems does the Food Ordering System depend on?

The Food Ordering System has four critical external dependencies:

1. **AWS Cognito** - Required for user authentication and authorization. Without this, users cannot log in or maintain secure sessions.
2. **Stripe** - Important for payment processing. The system cannot complete orders without payment capability.
3. **Push/Email Provider** - Necessary for customer communication for order status, [confirmations, updates].
4. **Loyalty Provider** - Integrates with the rewards program to track points and validate **loyalty-based** discounts.

3. How is the flow of interaction between the user and the system represented?

The interaction flow is represented through directional arrows that show the relationships, which are:

- **User → Food Ordering System:** Users initiate actions by placing orders, viewing menus, and tracking deliveries.
- **Admin → Food Ordering System:** Admins manage backend operations like inventory and disputes.
- **Food Ordering System → External Systems:** The system makes external calls to **authenticate** users using (**Cognito**), **process payments** (**Stripe**), **send notifications** (**Push Provider**), and **update rewards** (**Loyalty Provider**).

This arrow flow shows who initiates each interaction and establishes system boundaries.

4. Are there any actors or external systems missing or redundant?

I would probably list the Missing:

- **Delivery Driver/Service:** If the system handles delivery logistics, a delivery service or driver actor might be needed.
- **Restaurant/Kitchen System:** If this is a multi-restaurant platform, restaurants might be separate actors.
- **Analytics/Stats Platform:** For business analytics and reporting.

No Redundancies: All listed external systems serve separate purposes and are necessary for the system's core functionality.

5. How does this diagram help you understand system boundaries?

The Context Diagram shows:

- **Inside the Boundary:** All food ordering functionality (order management, inventory, payments, tracking, disputes, discounts, analytics).
- **Outside the Boundary:** Authentication (Cognito), payment processing (Stripe), notifications (Push Provider), and loyalty management (Loyalty Provider).

That separation helps us to recognize:

- What we control vs. what we depend on
- Integration points that require API calls
- Third-party dependencies that could affect system availability
- Areas where we need fallback strategies if external services fail

Applications Questions:

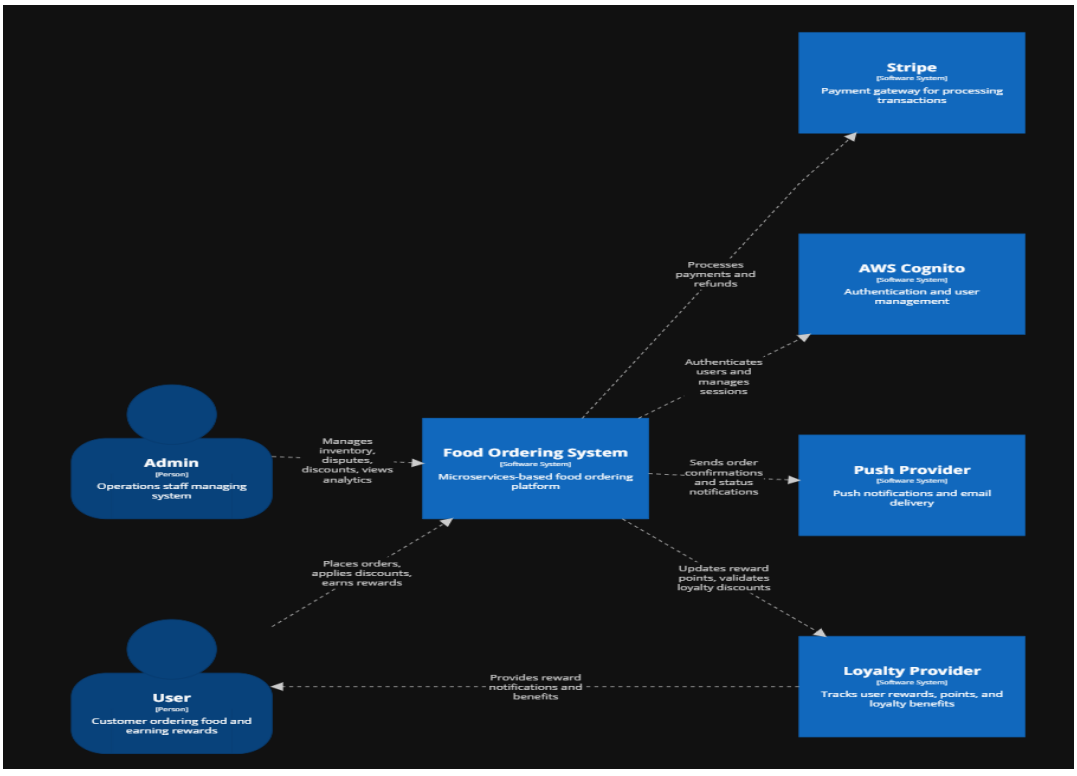
1. Add a new external system: “Loyalty Provider” that tracks user rewards. How would you represent it in the context diagram?

   I will define it as a softwareSystem, and add the name, and description .

   loyalty = softwareSystem "Loyalty Provider" "Tracks user rewards, points, and loyalty benefits"

2. Draw the relationship arrows to show how users and admins interact with this new system.

   user -> ss "Places orders"
   admin -> ss "Manages inventory, disputes"
   ss -> loyalty "Updates reward points, validates loyalty discounts"



Level 2 – Container Diagram (C4 L2)

1. Identify the **containers** (Web App, Mobile App, Backend services, Databases).

Monolithic Architecture Containers:

- **Web App:** React-based browser interface
- **Mobile App:** iOS/Android native applications
- **Monolithic Backend:** Single Node.js/Java application containing all business logic
- **Database:** Single PostgreSQL database storing all data

Microservices Architecture Containers:

- **Client Layer:** Web App (React), Mobile App (iOS/Android)
- **Gateway Layer:** API Gateway (Kong/NGINX/AWS API Gateway)
- **Service Layer:**
    - Order Service
    - Inventory Service
    - Payment Service
    - User Service
    - Notification Service
    - Tracking Service
    - Dispute Service
    - Discount Service (NEW)
    - Analytics Service (NEW)

- **Data Layer:** Separate database for each service (9 total PostgreSQL databases)

2. How are **Web and Mobile clients interacting** with backend containers?

Monolithic Architecture:

- Both Web App and Mobile App make direct HTTP/REST API calls to the single Monolithic Backend.
- All requests go through the same backend endpoint, which then routes internally to appropriate modules.

Microservices Architecture:

- Web App and Mobile App both call the **API Gateway** instead of services directly.
- The API Gateway acts as a single entry point, routing requests to appropriate microservices.
- This provides:
    - Centralized authentication/authorization
    - Load balancing
    - Rate limiting
    - Request routing based on URL patterns
    - SSL termination

3. What **data storage containers** exist, and which containers read/write to them?

Monolithic Architecture:

- **Single Database:** All modules (Order, Inventory, Payment, User, Notification, Tracking, Dispute) read/write to the same PostgreSQL database.

Microservices Architecture:

- Order Service →**Order DB**
- Inventory Service → **Inventory DB**
- Payment Service → **Payment DB**
- User Service →**User DB**
- Notification Service → **Notify DB**
- Tracking Service → **Tracking DB**
- Dispute Service → **Dispute DB**

4. Which containers are responsible for **external integration** (Stripe, Cognito, Push/Email)?

Monolithic Architecture:

- **Monolithic Backend** handles ALL external integrations:
  - **Stripe** for payments
  - **Cognito** for authentication
  - **Push Provider** for notifications

Microservices Architecture:

- **Each service is responsible for handling it's external integration:**
- **Payment Service** → Stripe (payment processing)
- **User Service** → Cognito (authentication)
- **Notification Service** → Push Provider (notifications)
- **User Service & Discount Service** → Loyalty Provider (reward validation and updates)

5. Compare Monolith vs Microservices: How do container responsibilities differ?

Aspect	Monolithic	Microservices
Responsibility Distribution	Single backend handles all business logic	Each service handles one business capability
Deployment	Deploy entire application as one unit	Deploy services independently
Scaling	Scale entire application (even if only one module needs more resources)	Scale individual services based on demand
Technology Stack	Must use same language/framework for all features	Each service can use different technologies
Database	Shared database creates coupling	Isolated and different databases ensure independence

Aspect	Monolithic	Microservices
Failure Impact	One bug can crash entire system	Service failures are isolated
Team Organization	Teams work on shared codebase	Teams own complete services end-to-end
Development Speed	Faster initially, slower as codebase grows	More overhead initially, scales better long-term

Application Questions

1. Extend the Microservices architecture with Discount Service and Analytics Service.

Discount Service:

- **Functionality:** Manages promotional codes, discount rules, coupon validation, and loyalty-based discounts.
- **Container:** "Discount Service", manages discounts, coupons, promotions
- **Database:** "Discount DB", stores discount rules, usage history, and validation data (PostgreSQL)
- **Technology:** Could use Node.js or Python for flexibility
- **Relationships:**
    - API Gateway → Discount Service (admin creates promotions, users validate codes)

- Order Service → Discount Service (validates and applies discounts during order creation)
- Discount Service → Loyalty Provider (validates loyalty-tier discounts)
- Discount Service → Analytics Service (sends discount usage events)
- Discount Service → Discount DB (reads/writes discount data)

Analytics Service:

- **Functionality:** Collects metrics from all services, processes data, and generates business analysis reports.
- **Container:** "Analytics Service", collects and processes system metrics
- **Database:** "Analytics DB", stores aggregated metrics, reports, and historical data (PostgreSQL, could also use time-series DB like TimescaleDB)
- **Technology:** Python for data processing, with libraries like Pandas, Numpy.
- **Relationships:**
  - API Gateway → Analytics Service (admins query reports and dashboards)
  - Order Service → Analytics Service (order events)
  - Payment Service → Analytics Service (transaction events)
  - Inventory Service → Analytics Service (stock events)
  - Discount Service → Analytics Service (discount usage events)
  - Analytics Service → Analytics DB (reads/writes metrics)

2. Represent the relationships of the new services to existing services and external systems.

Discount Service Relationships:

ss.apigw -> ss.discountSvc "Routes requests"

ss.orderSvc -> ss.discountSvc "Validates and applies discounts"

ss.discountSvc -> ss.discountDb "Reads/writes"

ss.discountSvc -> loyalty "Validates loyalty discounts"

ss.discountSvc -> ss.analyticsSvc "Sends discount usage events"

Analytics Service Relationships:

ss.apigw -> ss.analyticsSvc "Routes requests"

ss.orderSvc -> ss.analyticsSvc "Sends order events"

ss.paymentSvc -> ss.analyticsSvc "Sends payment events"

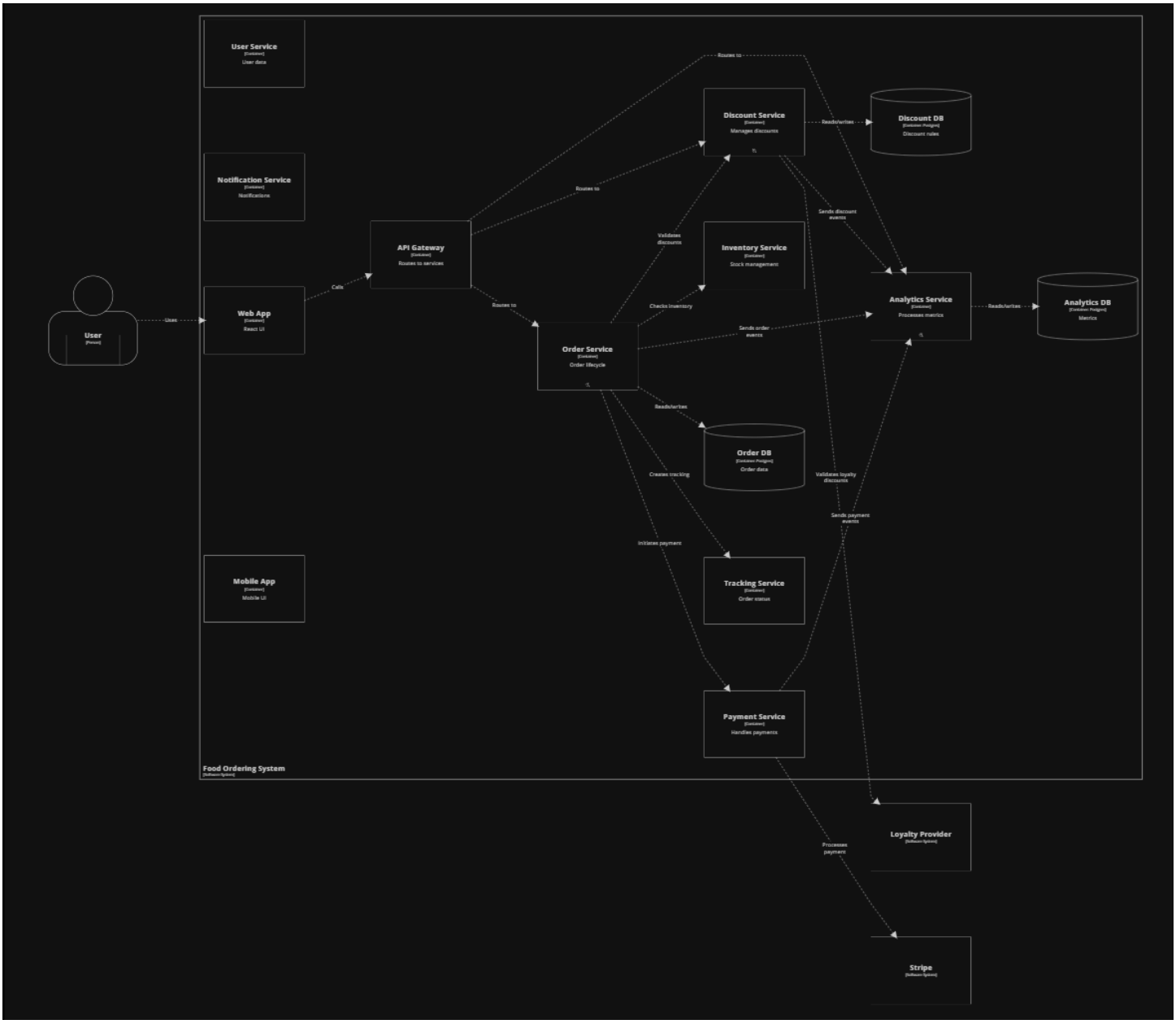
ss.inventorySvc -> ss.analyticsSvc "Sends inventory events"

ss.discountSvc -> ss.analyticsSvc "Sends discount usage events"

ss.analyticsSvc -> ss.analyticsDb "Reads/writes"

Continue to next page →

That’s what the system what will look like after adding the requirements (Analytics, Discount)



Level 3 – Component Diagram (C4 L3)

Observation / Analysis Questions:

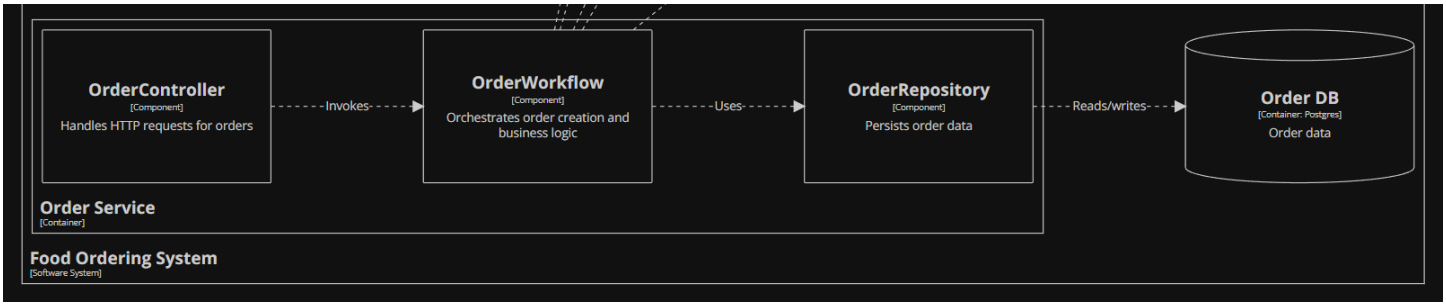
1. Pick a container (e.g., Order Service). Identify its **internal components**.

Let's take for example the Order Service:

It contains 3 components:

1. **OrderController:**
   - Handles incoming HTTP requests (REST endpoints)
   - Validates request parameters and authentication tokens
   - Returns HTTP responses with appropriate status codes
2. **OrderWorkflow:**
   - Contains business logic for order lifecycle
   - Orchestrates calls to other services (Inventory, Payment, Discount)
   - Implements order state transitions (Pending → Confirmed → Preparing → Delivered)
3. **OrderRepository:**
   - Abstracts database access
   - Performs CRUD operations on Order DB
   - Provides query methods (findByUserId, findByStatus, etc.)

2. How do components **communicate with each other** within the same container?



Let's follow that flow, first, the OrderController receives HTTP requests, then it validates and handles them and it calls the OrderWorkFlow, the WorkFlow runs business logic that validates discount via discount service, and checks inventory same as before, and then creates a record for the order via the repo

3. How do components interact with **databases** and **external systems**?

Only the Repository component interacts with the database, **Workflow components** make calls to external services.

OrderWorkflow → Discount Service (HTTP: validate discount)
OrderWorkflow → Inventory Service (HTTP: check stock)
OrderWorkflow → Payment Service (HTTP: process payment)
OrderWorkflow → Tracking Service (HTTP: create tracking)
OrderWorkflow → Analytics Service (Message: log event)

4. Which components are responsible for **business logic**, which are just **controllers or repositories**?

Controllers handle HTTP requests only, not business logic, also repositories doesn't handle business logic only like a data layer, so, there is Workflow only left, they're responsible for business logic and rules.

- Can you identify any **tight coupling** or potential areas of complexity?

Tight Coupling:

1. **OrderWorkflow's Multiple Dependencies:**
    - Depends on Discount Service, Inventory Service, Payment Service, Tracking Service, Analytics Service.
    - If any service changes its API contract, OrderWorkflow must be updated.
    - Creates a distributed monolith if not careful.

2. **Synchronous Service Calls:**
    - Order creation requires multiple synchronous calls in sequence
    - If Discount Service is slow (2 seconds), entire order creation is delayed
    - failures are possible if one service is down
3. **Shared Understanding of Domain Objects:**
    - Services must agree on Order structure, pricing rules, status codes
    - Changes to domain model require coordination across teams

Complexity Areas:

1. **Transaction Management:**
    - Creating an order involves multiple services and databases
    - What happens if payment succeeds but inventory update fails?
    - Requires distributed transaction coordination
2. **Analytics Collector Bottleneck:**
    - Receives events from many services synchronously
    - Can become a performance bottleneck
    - Single point of failure if it goes down
3. **Error Handling Complexity:** Must handle partial failures carefully

Application Questions:

1. Add **components** for the new Discount Service: Controller, Workflow, Repository.

DSL Code:

```
discountSvc = container "Discount Service" {

discountController = component "DiscountController" "Handles HTTP requests for discount validation and management"

discountWorkflow = component "DiscountWorkflow" "Validates discount rules, checks eligibility, applies business logic"

discountRepo = component "DiscountRepository" "Persists discount codes, rules, and usage history"

}
```

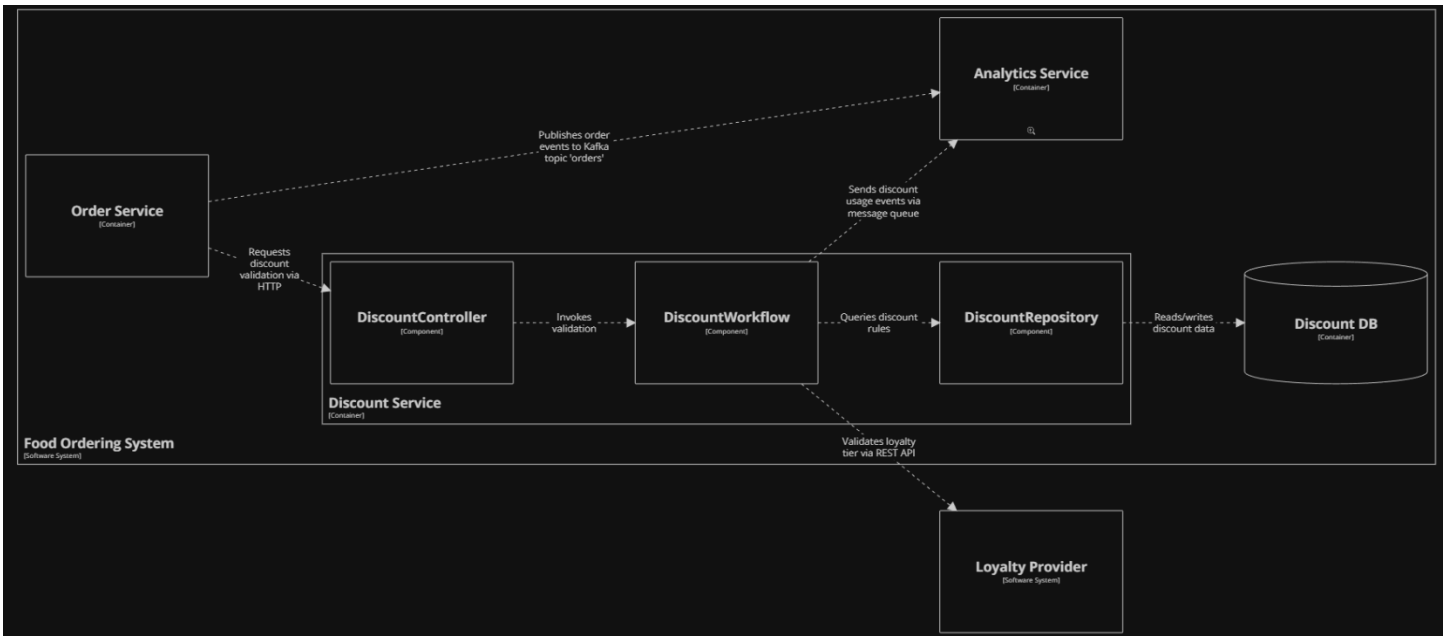
2. Add **components** for the new Analytics Service: Collector, Processor, Repository.

DSL Code:

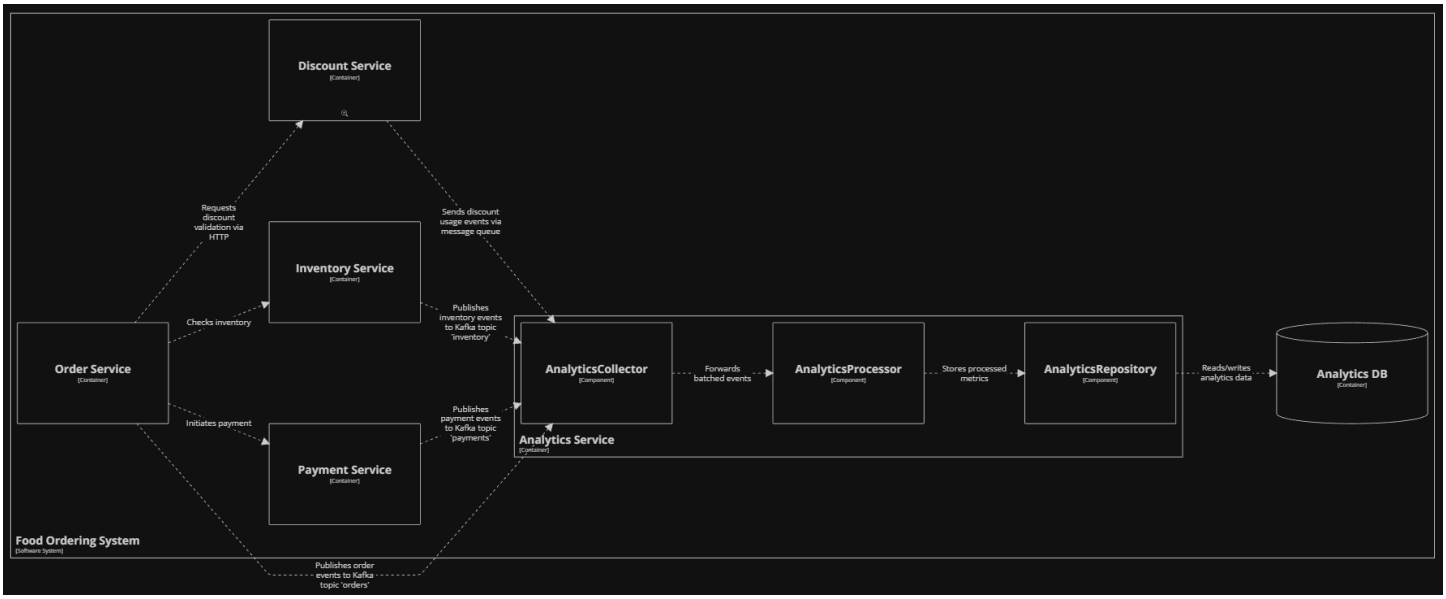
```
analyticsSvc = container "Analytics Service" {
 analyticsCollector = component "AnalyticsCollector" "Receives events from all services via REST or message queue"
analyticsProcessor = component "AnalyticsProcessor" "Aggregates metrics, generates reports, performs data analysis"
analyticsRepo = component "AnalyticsRepository" "Stores raw events and aggregated analytics data"
 }
```

3. Show the relationships of these components to OrderService, PaymentService, and databases.

Discount Service



Analytic Service



Let's start with **discount service**: **DiscountController** receives HTTP request from **Order Service**, **DiscountWorkflow** executes **business logic** (validation, calculation), **DiscountRepository** queries **database** for discount rules, **Discount DB** stores discount codes, rules, usage history.

Analytics service: **AnalyticsCollector** receives events from multiple services via Kafka, **AnalyticsProcessor** aggregates and processes events, **AnalyticsRepository** persists metrics, **Analytics DB** stores raw events and aggregated metrics

Cross-Level Reflection Questions

1. How do Context, Container, and Component diagrams **relate to each other**?

Let's say the C4 Model is like a map (Google Maps), each time you zoom in you see more details and things. Same is C4 Model.

You have:

Context level → Container Level → Component Level → Code Level.

From Context level you see how the user interacts with system, and system with other systems, when you zoom in that level to any software system or container, you will see smaller pieces (containers).

And each one is having a relationship with a container or system, internal or external system. Zoom more in you will see how each component is made of, how it interacts from the inside to outside.

Each level targets different audiences, the context targets most people, like: stakeholders, product managers, and container level targets architects, tech leaders, and component level targets mostly developers.

2. How does C4 help you **visualize responsibilities, boundaries, and dependencies**?

Ok, let's start with:

Responsibilities:

L1 shows what the system is responsible for and what external systems are responsible for, like Stripe handling payment processing.

L2 shows what each container responsible for and what is the main functionality for it, like order service container it has controller, repo components.

L3 shows that each component is responsible for and what behavior it does, like order controller handling HTTP requests.

Boundaries:

Each internal and external system are clearly separate and visualized; each service and system are visualized. That lets us spot tight coupling, architecture design issues, etc.

We can spot each component's job and resources, which enables us to see boundaries and authorities.

Dependencies:

In the C4 Model the arrows show the direction the dependencies, or any external system (Dependency).

System → Stripe

3. How would adding a new feature differ in Monolith vs Microservices at each level?

Let's assume a feature addition "Restaurant Reviews".

In Monolithic Approach:

L1: Just change the description, or maybe add some external system interaction (minimal changes).

L2: No new container is added (probably), there will be change in backend code and components, no edit in DB.

L3: Review Module is added (Controller, Service, Repo) and each component.

Microservices:

L1: Like Monolith, just changing description, adding external systems.

L2: There will be new containers added (Review Service), and its API gateway links and its Database.

L3: New components will be created in that container (controller, repo, workflow).

4. Which level is most helpful for **communication with non-technical stakeholders**?

Level 1 (System Context) is the choice, it uses simple, straight forward language for most people, it shows users actions, and shows external systems.

5. Which level is most helpful for **developers implementing the system**?

My opinion is **Level 3**, it shows actual code structure, you can see classes and modules, how these class and modules interact with each other. It makes testing easier, makes responsibilities clearer.

Don't forget to mention that **Level 2** is also important for understanding the system from a bigger picture view.

6. **Vertical vs Horizontal Scaling**

a. Identify which services should scale vertically vs horizontally.

Ok, let's put some dots on the letters first.

Vertical Scaling: Adding more power (CPU / RAM / Storage)

Horizontal Scaling: Adding more server (machine) instances.

Let's talk about **Database containers**, let's say **vertical** scaling is the way to go, most DB's benefits from high caching memory, resulting in faster queries. Also, you can say if you have distributed servers around the world you need also to scale **horizontally**, which is having more servers and machines around the world.

Analytics Service → <u>vertical scaling</u>; Data processing requires significant CPU and memory and Report generation is slow, large dataset processing**.**

API Gateway →<u>vertically</u>; routing benefit from faster CPUs, especially at high connection count. **Mixed approach** is also valid (vertical + horizontal) scaling.

Order Service → <u>horizontally</u> scaled; Stateless, can run multiple instances, at High request volume during lunch/dinner times. An important thing to take into consideration is **load Balancer.**

Notification Service → <u>horizontally</u> scaled; Independent notification requests at the same time, at high volume of notifications (promotions, order updates).

Payment Service → <u>horizontally</u> scaled; Stateless transaction processing at high transaction volume**.**

Discount Service → <u>horizontally</u> scaled; Read heavy validation requests, Promotion campaigns (everyone using discount codes)

Inventory Service → <u>Horizontally</u>; Many concurrent stock checks, at Popular items being ordered frequently

Web App / Mobile App → Horizontally, Stateless front-end servers, at High user traffic for, for example: CDN for Multiple regions for low latency.

Hybrid Approach (Both Vertical and Horizontal):

1. API Gateway:

- Horizontal: Multiple gateway instances across availability zones
- Vertical: More powerful machines for API processing

2. Inventory Service:

- Horizontal: Multiple service instances
- Vertical: Beefier database server for write performance

b. How does this differ in monolith vs microservices?

Monolithic Architecture Scaling:

Only Horizontal Scaling Option:

- Add more instances of the **entire monolith**
- Load balancer distributes requests across instances

Problem: Inefficient Resource Usage

- If only Payment Module is slow → must scale entire application
- If only Analytics Module needs memory → must scale everything
- Inventory, Orders, Notifications all get scaled unnecessarily

Microservices Architecture Scaling:

Selective Scaling:

- Scale only services under load
- Each service scales independently

Independent Resource Allocation:

- **Payment Service:** High-CPU instances (fast transaction processing)
- **Analytics Service:** High-memory instances (data aggregation)
- **Notification Service:** Many small instances (distributed load)
- **Inventory Service:** Medium instances with SSD (fast lookups)

Database Scaling:

- Each service chooses appropriate strategy
- **Order DB:** Read replicas for scaling reads
- **Analytics DB:** Time-series database with partitioning
- **Inventory DB:** Vertical scaling + caching layer
- **Discount DB:** Redis for fast lookups

Aspect	Monolithic	Microservices
Scaling Granularity	All-or-nothing	Per-service
Resource Efficiency	Low (waste on unused modules)	High (pay for what you need)
Scaling Strategy	Horizontal only (application)	Horizontal + Vertical (per service)
Instance Types	One size for all	Optimized per service
Autoscaling	Based on overall load	Based on per-service metrics
Database Scaling	Limited options	Multiple strategies
Cost During Peak	High	Moderate (scale what's needed)
Cost During Off-Peak	High (can't scale down much)	Low (big scale down)
Operational Complexity	Low	High (many services to monitor)

7. In the Monolithic architecture (Level 2), all external dependencies (Stripe, Cognito, Push Provider) are connected to the single Monolithic Backend container. In the Microservices architecture (Level 2), each dependency is connected to a specific service (e.g., Stripe to Payment Service). What is the primary benefit of the microservices approach in this regard?

The primary Benefit: Separation of Concerns and Fault Isolation (Single point of failure).

1. Fault Isolation

Monolith:

- Stripe API down → entire backend might hang waiting for timeout
- Cognito slow → all requests delayed (authentication blocks everything)
- Single point of failure affects all functionality

Microservices:

- Stripe API down → only Payment Service affected
- Users can still: browse menu, add to cart, track orders, view history
- Other services continue operating normally

2. Independent Updates and Maintenance

Monolith:

- Stripe announces new API version with breaking changes
- Must update entire application
- Full regression testing required
- All developers must stop working

- Risky deployment

Microservices:

- Only Payment Service team updates Stripe integration
- Other teams continue normal development
- Independent testing of Payment Service
- Deploy only Payment Service (low risk)
- Gradual rollout possible

Specialized Error Handling

Microservices Advantage:

Each service implements error handling specific to its external dependency

Monolith:

- Generic error handling for all external dependencies
- Can't optimize for specific service characteristics
- Complex code trying to handle all cases

Performance Optimization

Each service can optimize its external calls independently

Team Ownership and Vendor Relationships

Microservices Benefits:

- **Payment team** owns complete Stripe relationship
    - Negotiate rates and fees
    - Direct contact with Stripe support
    - Deep expertise in payment processing

- **User team** owns Cognito integration
    - Understands authentication patterns
    - Optimizes user onboarding flow

Monolith:

- Shared responsibility
- "Who should I ask about Stripe errors?" → Unclear
- Fragmented Vendor relationships

7. Technology Flexibility

Microservices:

- Payment Service: Use Stripe's Node.js SDK (best support)
- User Service: Use AWS Cognito SDK for Python (team expertise)
- Notification Service: Use FCM SDK for Go (performance)

Monolith:

- Must use same language for all integrations
- Can't leverage best SDK for each service
- Limited by initial technology choice

8. Resilience Engineering Question

If the Inventory Service becomes unavailable, how does Order Service respond?

Should there be:

a. a fallback,

b. a cached inventory,

c. a queue,

d. or transaction compensation?

Answer: Hybrid Approach, a mix of all options.

Layered Resilience

Layer 1: Cached Inventory (Option b) → First Line of Defense

Benefits:

- Cache hit in neglectable time
- Can process orders in times of outage
- **Reduces load** requests

Limitations:

- **Stale data risk:** Cache might be old
- **Cache invalidation:** Hard to keep perfectly synchronized

When to use:

- High-traffic scenarios
- Inventory changes slowly
- Acceptable to have slightly stale data

Layer 2: Queue (Option c)

Benefits:

- **Survives long outages:** Orders queued until service recovers
- **No data loss:** All orders eventually processed
- **Automatic retry:** Worker retries until success
- **Customer not blocked:** Gets immediate response

Trade-offs:

- **Delayed confirmation:** Customer doesn't know immediately if order succeeded
- **Possible cancellations:** Order might be cancelled hours later
- **Customer experience:** "Your order is being verified..."

When to use:

- Services with occasional long outages
- Non-urgent orders (delivery tomorrow)
- Eventual consistency acceptable

Layer 3: Transaction Compensation (Option d)

Benefits:

- **Clean failure:** User knows order didn't go through
- **No dirty data:** Everything rolled back

Limitations:

- **User frustration:** Order failed, must retry manually
- **Lost opportunity:** Customer might abandon cart
- **Compensation complexity:** Must implement reverse operations

When to use:

- Strong consistency required
- Financial transactions (can't have ambiguous state)
- User expects immediate definitive answer

9. Retry, Idempotency, Timeout Strategy

For each connector that performs a potentially state-changing operation (e.g., create order), specify:

- Retry rules
- Timeout settings
- Idempotency keys

Represent these decisions at the connector level.

Connector 1: Order Service → Payment Service (Process Payment)

Operation: POST /payments/charge

Timeout:

- **Connection timeout:** 3 seconds
- **Read timeout:** 10 seconds
- **Total timeout:** 15 seconds
- **Rationale:** Payment processing involves external API (Stripe), must wait for response but not forever. User expects payment to complete within reasonable time.

Retry Rules:

- **Retry on:** Network errors (connection refused, timeout), 5xx server errors (503 Service Unavailable, 500 Internal Server Error)
- **Do NOT retry on:** 4xx client errors (402 Payment Required/card declined, 400 Bad Request)
- **Max retries:** 2
- **Circuit breaker:** Open after 5 consecutive failures, half-open after 30s

- **Rationale:** Payment operations are expensive; retry only transient failures. Too many retries risk duplicate charges.

Connector 2: Order Service → Inventory Service

Operation: POST /inventory/reserve

Timeout:

- **Connection timeout:** 2 seconds
- **Read timeout:** 5 seconds
- **Total timeout:** 8 seconds
- **Rationale:** Inventory checks should be fast database operations. Longer timeouts indicate systemic problems.

Retry Rules:

- **Retry on:** Network errors, 503 Service Unavailable, timeout
- **Do NOT retry on:** 409 Conflict (stock already reserved for this order), 404 Not Found (item doesn't exist), 410 Gone (item discontinued)
- **Max retries:** 3
- **Rationale:** Quick retries acceptable since operation is lightweight. Jit

Connector 3: Order Service → Notification Service (Send Confirmation)

Operation: POST /notifications/send

Timeout:

- **Connection timeout:** 2 seconds
- **Read timeout:** 5 seconds
- **Total timeout:** 8 seconds
- **Rationale:** Notifications are non-critical; don't block order flow for notification failures.

Retry Rules:

- **Retry on:** All errors (network, 4xx, 5xx)
- **Max retries:** 5 (aggressive since non-critical)
- **Async:** Use message queue for retries (don't block order creation)
- **Rationale:** Notifications should eventually be delivered; Use queue so retries don't delay order response.

Connector 4: Order Service → Discount Service (Validate & Apply Discount)

Operation: POST /discounts/apply

Timeout:

- **Connection timeout:** 1 second
- **Read timeout:** 3 seconds
- **Total timeout:** 5 seconds
- **Rationale:** Discount validation is **synchronous** (blocks order creation); must be **fast** or **fail** quickly.

Retry Rules:

- **Retry on:** Network timeout, 503 Service Unavailable (service overloaded)
- **Do NOT retry on:** 404 (code not found), 400 (invalid code), 410 (expired), 429 (rate limited)
- **Max retries:** 2
- **Rationale:** Fast operation; immediate retry catches network glitches. Must not delay order.

Connector 5: Payment Service → Stripe (External API)

Operation: POST

Timeout:

- **Connection timeout:** 5 seconds
- **Read timeout:** 15 seconds
- **Total timeout:** 20 seconds
- **Rationale:** External API; must accommodate network latency and Stripe's processing time. Stripe recommends 80s but we use 20s for better UX.

Retry Rules:

- **Retry on:** Network errors , 500 Internal Server Error, 503 Service Unavailable from Stripe
- **Do NOT retry on:** 402 (card declined), 400 (invalid parameters), 401 (authentication failed), 429 (rate limit)
- **Max retries:** 2
- **Rationale:** Stripe is reliable; excessive retries indicate systemic issues. Respect their rate limits.

Connector 6: User Service → AWS Cognito (Authentication)

Operation: POST (InitiateAuth)

Timeout:

- **Connection timeout:** 3 seconds
- **Read timeout:** 7 seconds
- **Total timeout:** 10 seconds
- **Rationale:** User login should be fast; longer delays frustrate users. Cognito is generally fast (<1s).

Retry Rules:

- **Retry on:** Network errors, 500/503 from Cognito (rare), TooManyRequestsException
- **Do NOT retry on:** NotAuthorizedException (invalid credentials), UserNotFoundException, PasswordResetRequiredException
- **Max retries:** 1 (authentication failures usually aren't transient)
- **Fallback:** If Cognito down, check cached session tokens
- **Rationale:** Wrong password won't be fixed by retry. Only network issues warrant retry.

Summary Table: All Connectors

Connector	Timeout	Max Retries	Critical?
Order → Payment	15s	2	Critical
Order → Inventory	8s	3	Critical
Order → Notification	8s	5	Non-critical
Order → Discount	5s	2	Semi-critical
Payment → Stripe	20s	2	Critical
User → Cognito	10s	1	Critical

Finally, I hope that you've enjoyed reading the assignment

Thanks for reading